

Collection Framework Type → Generic
Type (variable হিসাবে use করা হয়।

Wrapper class use করতে হবে (Generics এর জন্য)

Generic is Type parameter variable

Is a way to create classes, interfaces, & methods that can operate on different types of data while maintaining type safety.

Generic Types → before type

Generic Method → before return type

```
class Test <T> {  
    T obj;  
    Test (T obj) {  
        this.obj = obj;  
    }  
    public T getObject() {  
        return this.obj;  
    }  
}
```

not mandatory

```
public class Main {
    psvm(...) {
```

```
        Test<Integer> intObj = new Test<>(100);
        s.out("Integer value: " + intObj.getObject());
```

```
        Test<String> strObj = new Test<>("Java");
        s.out("String: " + strObj.getObject());
```

```
    }
```

```
}
```

```
class A<T> {
```

```
    T a;
```

```
    A(T a) { this.a = a; }
```

```
    void Show() { s.out("Value of a: " + a); }
```

```
}
```

```
public class Main {
```

```
    psvm(...) {
```

✓ wrapper class

```
        A<String> ob1 = new A<>("A+");
```

```
        ob1.Show();
```

```
    }
```

```
}
```

class A<A1, A2> | A<T1, T2>{

T1 a;

T2 b;

A (T1 a, T2 b){

this.a = a;

this.b = b;

}

...

}

A <Character, String>

[This is decided during Compile Time]

Can use custom types

#Generics w/ Interface

Shayan

Interface can be Generic too!

- ☑ abstract
- ☑ no body, method signature
- ☑ method override with public

```
interface Processor <T> {  
    void process (T value);  
}
```

```
class StringProcessor implements  
    Processor <String> {  
    public void process (String value) {  
        S.out(" Process: " + value);  
    }  
}
```

```
class IntegerProcessor implements  
    Processor <Integer> {  
    public void process (Integer value) {  
        S.out (" Process: " + value);  
    }  
}
```

Public class Main {
 psvm(...) {

Shayan

Processor <String> P = new StringProcessor();
 P.process("Java");

Processor <Integer> K = new IntegerProcessor();
 K.process(5);

}

}

class StringProcessor <T> implements Processor <T> { ... }

psvm(...) {

Processor <String> ob1 = new StringProcessor <> ();

}

#Generic Method

```
class A {  
    <T> void search(T[] a) {  
        for (int T b : a)  
            s.out(b);  
    }  
}
```

```
public class Main {  
    psum(...) {  
        A ob1 = new A();  
        int a[] = {1, 2, 3};  
        String b[] = {"AB", "CD", "EF"};  
        ob1.search(a);  
        ob1.search(b);  
    }  
}
```



```
class Inspector {
```

```
<T> void search(T[] arr, T target) {
```

```
    int len = arr.size();
```

```
    for (int i = 0; i < len; i++) {
```

```
        if (arr[i] == target) {
```

```
            System.out.println("Target found.");
```

```
            System.out.println("Index : " + i);
```

```
            return;
```

```
        }
```

```
    }
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        String[] arr = { "AB", "CD", "EF" };
```

```
        Inspector is = new Inspector();
```

```
        is.search(arr, "CD");
```

```
    }
```

```
}
```

Lab Exam (CSE342)

- ① Exception
- ② Collection Framework
- ③ Multithreading
- ④ Generics
- ⑤ String Operations

12) A university stores grades of different types :

Integer (Marks)

Double (CGPA),

String (Grade such as A, B, C).

Create a generic class `Grade<T>`, store the grade of any type & display the grade.

```
class Grade<T> {  
    T grade;  
    Grade(T grade) {  
        this.grade = grade;  
    }  
    void show() {  
        System.out.println("Grade: " + this.grade);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Grade<Integer> g1 = new Grade<>(4);  
        Grade<Double> g2 = new Grade<>(3.98);  
        Grade<String> g3 = new Grade<>("A+");  
        g1.show();  
        g2.show();  
        g3.show();  
    }  
}
```